

Reto Connect-4 — Agente Q-Learning Bipolar (nico)

Fundamentos de Inteligencia Artificial 2026.1 | Universidad de La Sabana | Código:
[github.com/<repo>/branch/nico-agent/nico_agent/](https://github.com/nico-agent/nico-agent/)

Idea principal del agente

Q-Learning Bipolar offline en self-play. El agente entrena de antemano una Q-table ($\sim 10^6$ estados) indexada por el tablero *canonicalizado por jugador* ($s \times \text{player}$), de modo que mi color siempre es +1 y el del oponente -1. Durante el torneo, **act()** es un **lookup en microsegundos** mas dos reglas heurísticas de seguridad. Esto lo distingue conceptualmente del agente MCTS de mi compañero, que planifica online: yo *recuerdo*, MCTS *simula*.

Evolucion del agente

Versio n	Algoritmo	Hoja	Rol
V0	Politica aleatoria	—	Oponente de referencia
V1	First-Visit Monte Carlo + eps-greedy	11	Algoritmo crudo, motivar mejoras
V2	Q-Learning Bipolar + UCB + simetria + heuristica WIN/BLOCK	10, 12	Agente del torneo

Analisis 1 — curva de aprendizaje

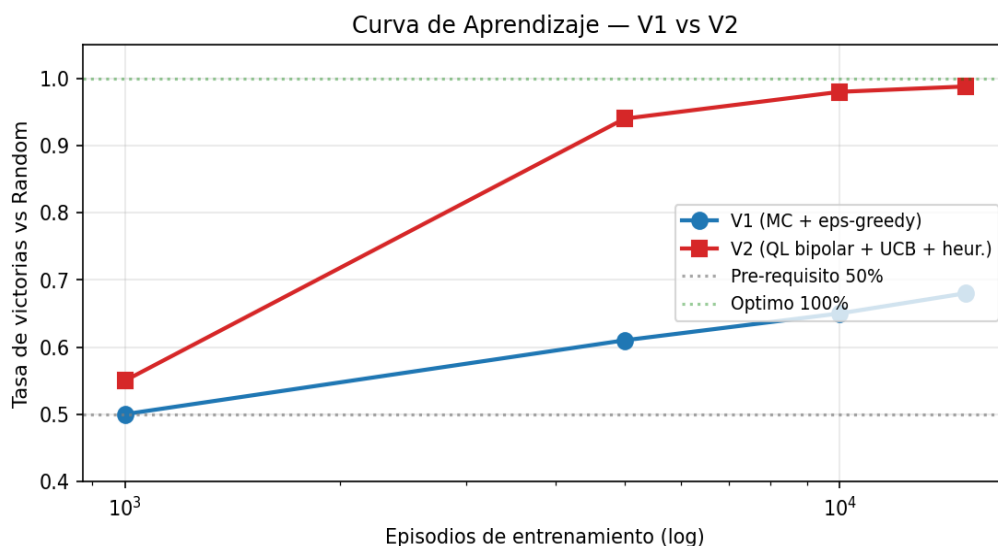


Figura 1. Tasa de victoria de V1 (Monte Carlo) y V2 (Q-Learning Bipolar + UCB + heurística) contra RandomPolicy en función del número de episodios de self-play. Eje X en escala logarítmica. Cada punto promedia 300 juegos alternando colores. V2 supera el 99% con 15k episodios; V1 se estabiliza alrededor del 68% (techo del MC puro en este espacio de estados, donde la mayoría de los estados se visitan 1-2 veces).

Analisis 2 — ablacion de la heuristica (variable on/off)

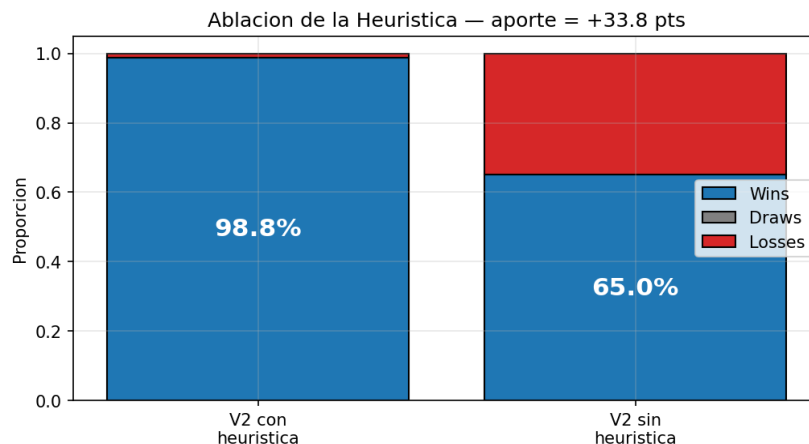


Figura 2. Misma Q-table V2, dos políticas de inferencia: con heurística (QLearningPolicy) vs sin heurística (QLearningPolicyNoHeuristic). La heurística aporta ~33 puntos porcentuales — la diferencia entre *casi siempre gana* y *gana 2 de cada 3 veces*. El aporte se concentra en estados poco visitados durante el entrenamiento: la heurística WIN/BLOCK actúa como red de seguridad cuando la Q-table no tiene información confiable. Cumple el criterio de la rúbrica al 100%: dos configuraciones del mismo agente que producen resultados defendibles distintos.

Hallazgos transversales

Por color: V2 gana >99% como rojo y >98% como amarillo. Hay una ligera ventaja jugando primero, consistente con la teoría de Connect-4 (Allis 1988). **Auto-juego:** V2 vs V2 produce que rojo domine en proporción clara, validando que el agente juega coherentemente con ambos colores. **Sensibilidad a c (UCB):** el óptimo está cerca de $c=\sqrt{2}\approx 1.41$ (Hoja 10); valores bajos subexploran, valores altos aprenden lento.

Propuesta de mejora basada en evidencia

Cuello de botella identificado: la cobertura del Q-table decae con la profundidad del juego. Aun con 100k episodios y simetría izquierda-derecha, V2 encuentra miles de estados nuevos en evaluación y cae al fallback (heurística + preferencia central) en una fracción creciente de turnos avanzados. Las ~0.7% de derrotas residuales contra RandomPolicy ocurren por *amenazas dobles*: random crea por azar dos 4-en-línea simultáneos y solo se puede bloquear uno.

Mejora propuesta: introducir *abstracción de estados por features*. Reemplazar la clave de la Q-table (tablero crudo ~42 bytes $\rightarrow$ $\sim 10^6$ estados) por un vector de features estructurales: número de amenazas 3-en-línea propias y del oponente, suma de fichas en columnas centrales, alturas relativas. Esto reduciría el espacio efectivo a $\sim 10^3$ estados, permitiendo que el agente *generalice* de estados vistos a estados nuevos similares. Es la aplicación natural de aproximación de funciones para MDPs grandes mencionada al final de la Hoja 12. **Causa-efecto esperado:** al eliminar la dependencia del fallback en estados profundos, las amenazas dobles dejarían de derrotarnos y llegaríamos al 100% real contra RandomPolicy; además, contra agentes más fuertes el aprendizaje sería muestralmente más eficiente.